

END-TO-END PROJECT

Retail Analytics Platform

AWS S3 · Snowflake · dbt · Apache Airflow · Medallion Architecture

Architecture · Design Decisions · Interview Q&A;

Project Overview — Retail Analytics Platform

This project builds a production-grade data platform for a retail company that processes daily sales data from 500 stores across four regions. CSV files land in AWS S3 every hour. The platform transforms raw data through the Medallion architecture — Bronze, Silver, Gold — using Snowflake as the cloud warehouse, dbt for SQL transformations, and Apache Airflow for pipeline orchestration.

Business Context — Stakeholder Requirements

Stakeholder	Requirement	How Delivered
Finance VP	Daily revenue by region by 7am	Gold DAILY_SALES_SUMMARY table refreshed by 5am via Airflow DAG
Marketing	Customer segmentation for campaigns	Gold CUSTOMER_360 table with RFM scoring, refreshed daily
Data Science	Clean historical data for ML	Silver SCD2 dimensions and conformed facts with full history
Operations	Pipeline health visibility	Airflow Gantt chart + Snowflake ACCOUNT_USAGE monitoring queries
Compliance	GDPR lineage and PII masking	dbt docs lineage graph + Snowflake dynamic masking policies

Tech Stack — Why Each Tool

Tool	Role	Why This Tool — Interview Answer
AWS S3	Landing zone for CSV files	Infinitely scalable object storage. Native Snowflake integration via Storage Integration (IAM role trust — no hardcoded keys). Files are immutable — never modify source data.
Snowflake	Cloud data warehouse	Decoupled storage and compute = independent scaling. Native CDC via Streams. SCD2-friendly with Time Travel and zero-copy cloning. Multi-cluster warehouses for concurrent BI access.
dbt	SQL transformation layer (the T in ELT)	Version-controlled SQL with automatic DAG management, built-in testing, auto-generated documentation, and environment isolation via profiles and targets.
Apache Airflow	Pipeline orchestration and scheduling	DAG-based scheduling with explicit dependency management, per-task SLA callbacks, Slack/PagerDuty alerting, and retry logic. Official operators for Snowflake and dbt.
Medallion Architecture	Data layer pattern	Bronze preserves raw data (reproducible from S3). Silver applies governed business rules. Gold serves pre-aggregated BI-ready data. Each layer independently rebuildable.

Complete Data Flow — Source to BI

END-TO-END DATA FLOW

SOURCE SYSTEMS → 5 CSV files generated hourly by order management, CRM, POS
orders.csv | customers.csv | products.csv | order_items.csv | stores.csv

AWS S3 LANDING BUCKET

s3://retail-platform-bucket/raw/ (5 prefixes, one per source)

S3 ObjectCreated event → SQS queue → Snowpipe AUTO_INGEST

BRONZE LAYER (BRONZE_DB.RAW — transient tables)

All columns VARCHAR · Row hash MD5 for CDC · _IS_PROCESSED flag

Snowflake Streams capture every INSERT → CDC classification

SILVER LAYER (SILVER_DB — dbt models)

Staging: type-cast, rename, deduplicate · Snapshots: SCD2 dimensions

Incremental facts with point-in-time FK resolution

Project Scope — Objects Created

Component	Count	Objects	Key Concepts
AWS S3	1 bucket, 5 prefixes	IAM role, SQS, event notifications	IAM trust policy, no hardcoded credentials, SQS ARN for Snowpipe
Snowflake	5 databases, 15 schemas	6 warehouses, 8 roles, 5 stages	Storage integration, file formats, RBAC, resource monitors
Bronze Layer	5 tables	3 Snowpipes, load audit log	COPY INTO, Snowpipe AUTO_INGEST, row hash CDC, transient tables
CDC Engine	4 streams	CDC_STATE, WATERMARK, EVENT_LOG	Stream-based + hash-based CDC, change classification, offset mechanics
dbt Staging	5 models	sources.yml, staging.yml	source(), ref(), type casting, deduplication with QUALIFY
dbt Silver	5 models, 2 snapshots	3 SCD2 dims, 1 SCD1, 2 facts	dbt snapshot for SCD2, incremental models, point-in-time FK
dbt Gold	4 models, 2 views	RFM macro, materialized view	Jinja macros, window functions, incremental Gold aggregations
dbt Tests	18 schema, 6 singular	dbt-expectations tests	Generic tests, singular tests, test severity, source freshness
Airflow DAG	1 DAG, 11 tasks	SLA callbacks, Slack alerts	SnowflakeOperator, BashOperator, SLA miss callback, XCom
Security	4 masking, 2 row access	Network policy, PII tags	Dynamic masking, territory row access, GDPR patterns

CHAPTER 1

AWS S3 — Landing Zone

Bucket structure · IAM trust · Snowpipe integration

1.1 S3 Bucket Structure

The S3 bucket is organized with one prefix per source table. Snowpipe reads from the raw/ prefix. Processed files age into archive/ after 90 days. dbt Slim CI artifacts (manifest.json) are stored in dbt-artifacts/ for state comparison between runs.

S3 Bucket Layout

```
retail-platform-bucket/
  raw/orders/           ← orders_YYYYMMDD_HHMMSS.csv.gz
  raw/customers/        ← customers_YYYYMMDD.csv.gz (daily full extract)
  raw/products/         ← products_YYYYMMDD.csv.gz
  raw/order_items/      ← order_items_YYYYMMDD_HHMMSS.csv.gz
  raw/stores/           ← stores_YYYYMMDD.csv.gz (weekly)
  archive/YYYY/MM/DD/   ← files moved here after 90 days
  dbt-artifacts/prod/   ← manifest.json for Slim CI
```

1.2 IAM Setup — No Hardcoded Credentials

Snowflake creates an IAM identity (user ARN + external ID) when you run CREATE STORAGE INTEGRATION. You paste those two values into an AWS IAM role trust policy. Snowflake then assumes that role to access S3 — no access keys stored anywhere.

Step	Action	Location
1	Run CREATE STORAGE INTEGRATION in Snowflake	Snowflake SQL worksheet
2	Run DESC INTEGRATION to get IAM_USER_ARN + EXTERNAL_ID	Snowflake — copy both values
3	Create IAM Role in AWS — 'Another AWS account' trust type	AWS IAM Console
4	Paste IAM_USER_ARN into Principal + EXTERNAL_ID into Condition	AWS IAM trust policy JSON
5	Attach S3 read policy to the IAM role (GetObject + ListBucket)	AWS IAM permissions
6	Update STORAGE INTEGRATION with the new role ARN	Snowflake SQL
7	Grant USAGE on integration to ETL roles	Snowflake SQL

KEY INSIGHT

IAM role trust = Snowflake assumes the role temporarily using STS. To revoke access instantly: remove the trust policy

1.3 Snowpipe AUTO_INGEST — How It Works

Snowpipe with AUTO_INGEST watches a stage and loads files within 1–5 minutes of arrival.

Component	Description	Interview Point
External Stage	Points to S3 prefix using Storage Integration	Stage references integration — no credentials in stage definition
Snowpipe	Holds the COPY INTO definition + AUTO_INGEST=TRUE	SHOW PIPES gives notification_channel = the SQS ARN to paste into AWS
S3 Event Notification	ObjectCreated → SQS → Snowpipe polls SQS	Must match prefix path in stage URL exactly
Load Deduplication	64-day ETag history prevents duplicate loads	Same as COPY INTO — FORCE=TRUE overrides but risks duplicates
Error Integration	SNS integration for Snowpipe load failure alerts	Separate from S3 event SQS — this is an outbound Snowflake alert

CHAPTER 2

Snowflake — Warehouse Layer

Databases · Warehouses · RBAC · CDC Streams

2.1 Medallion Database Architecture

Database	Purpose	Retention	Table Type
BRONZE_DB	Raw landing — exact source copy	1 day	Transient (no Fail-safe)
SILVER_DB	Typed, SCD2 dims, conformed facts	30 days	Permanent
GOLD_DB	Business aggregations, BI-ready	14 days	Permanent
COMMON_DB	Shared: stages, file formats, UDFs, audit	7 days	Permanent
CDC_DB	Streams, control tables, CDC staging	7 days	Mix

2.2 Warehouse Strategy

Warehouse	Size	Auto-Suspend	Used For	Sizing Reason
INGEST_WH	X-Small	60 sec	COPY INTO, Snowpipe catch-up	COPY INTO is I/O-bound not compute-bound — X-Small is sufficient
TRANSFORM_WH	Medium	120 sec	dbt models, CDC processing	Balance speed vs cost for ELT transforms
ANALYTICS_WH	Medium multi-cluster	180 sec	BI tools, analyst queries	Multi-cluster handles 50+ concurrent dashboard users
AIRFLOW_WH	X-Small	60 sec	Airflow SnowflakeOperator calls	Lightweight SQL calls — no heavy compute needed
DATA_SCIENCE_WH	Large Snowpark-Opt	300 sec	Python/ML workloads	Snowpark-Optimized: 16× memory per node for pandas and ML
DEV_WH	X-Small	30 sec	Developer sandbox	Aggressive 30-sec suspend minimises idle cost in dev

2.3 RBAC Role Hierarchy

Roles form a hierarchy — granting a child role to a parent means the parent inherits all child privileges automatically. The four required grants for ANY query are: DATABASE USAGE, SCHEMA USAGE, TABLE SELECT, WAREHOUSE USAGE.

Role	Warehouse Access	Data Access	Purpose
ETL_ADMIN_ROLE	All warehouses	All DBs full	Pipeline administration
DATA_ENG_ROLE	TRANSFORM + DEV	All DBs read/write	Data engineering work
ETL_SVC_ROLE	INGEST + TRANSFORM	Bronze insert, Silver write	Service account for pipeline
CDC_SVC_ROLE	TRANSFORM	Bronze read, CDC_DB all	CDC engine service account
SR_ANALYST_ROLE	ANALYTICS	Silver + Gold read	Senior analysts
ANALYST_ROLE	ANALYTICS	Gold read only	Business analysts
REPORT_VIEWER_ROLE	ANALYTICS	Gold secure views only	BI tool service account

2.4 CDC — Four Patterns

Change Data Capture identifies only rows that changed since the last run. For 1 million customers with 5,000 daily changes, CDC processes 5K rows instead of 1M — 200× more efficient.

Pattern	Mechanism	Used For in This Project	Key Trade-off
Stream-based (native)	Snowflake Stream — METADATA dollar ACTION and METADATA dollar ISUPDATE track INSERT / UPDATE / DELETE on Bronze tables	Customers, Products, Employees dimensions	Exactly-once, native — but stream must be consumed before STALE_AFTER
Hash-based	MD5 of all columns computed at COPY INTO time. Compare current hash to last known hash in CDC_STATE control table.	Full-file extracts from legacy systems (no updated_at column)	Works universally — adds MD5 compute overhead at load time
Timestamp watermark	Process rows where _LOAD_TS is greater than last successful run timestamp stored in PIPELINE_WATERMARK control table.	Orders — append-only, never updated	Simple but misses deletes — combine with hash for completeness
Log-based (Debezium)	Reads source DB transaction log via Debezium → Kafka Connector	Not implemented — shown conceptually for real-time use cases	Sub-second latency — requires Kafka infrastructure

IMPORTANT

Stream offset only advances when the DML transaction that CONSUMES the stream commits. A failed Airflow task does not

CHAPTER 3

dbt — Bronze to Silver

Staging · Snapshots (SCD2) · Incremental facts · Tests

3.1 dbt Project Layer Structure

Layer	Folder	Materialization	Naming	Purpose
Staging	models/staging/	view	stg_tablename	Type-cast, rename, deduplicate — one model per source table
Intermediate	models/intermediate/	view	int_description	Cross-source joins, business logic — no final dim/fact yet
Dimensions	models/marts/dimensions/	table or snapshot	dim_name	Conformed dimensions — SCD2 via dbt snapshot
Facts	models/marts/facts/	incremental	fct_name	Grain-level facts — point-in-time FK to SCD2 dims
Gold	models/marts/gold/	incremental	(descriptive)	Pre-aggregated KPIs, RFM, Customer 360

3.2 Staging Model — Key Patterns

A staging model does exactly three things: type-cast all columns to proper types, rename columns to the project naming convention, and deduplicate using QUALIFY. Never apply business logic in staging.

stg_orders.sql — Pattern Summary

```
-- Materialization: view (no storage cost for staging)
-- Source: Bronze_DB.RAW.RAW_ORDERS via source('bronze','raw_orders')
-- Key transforms applied per column:
ORDER_ID      VARCHAR cast + NOT NULL filter
ORDER_DATE    TRY_TO_DATE with format 'YYYY-MM-DD'
STATUS        UPPER + TRIM for consistency
TOTAL_AMOUNT  TRY_TO_NUMBER with fallback COALESCE(x, 0)
NET_AMOUNT    Derived: total_amount * (1 - discount_pct/100)
-- Deduplication:
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id
                           ORDER BY _load_ts DESC) = 1
```

3.3 dbt Snapshots — SCD Type 2

dbt snapshots implement SCD2 automatically. When a tracked column changes, dbt adds an end date to the old row and inserts a new current row. Four columns are added automatically: dbt_scd_id, dbt_updated_at, dbt_valid_from, dbt_valid_to.

Config Option	Value Used	Meaning
strategy	timestamp	Use an updated_at column to detect changes
updated_at	updated_at	Source column that changes on every UPDATE

unique_key	customer_id	Business key — same across all versions
target_schema	snapshots	Schema where snapshot table is created
check strategy (alt)	check_cols list	Use when source has no updated_at — hashes specified columns

KEY INSIGHT

To query the current version of a snapshot: `WHERE dbt_valid_to IS NULL`.

3.4 Incremental Fact — Point-in-Time FK Resolution

The fact model is incremental (processes only new orders each run) and uses point-in-time FK resolution to join to the dimension version active when the order was placed.

fct_orders.sql — Critical Patterns

```
-- Config: materialized=incremental, unique_key=order_id
-- cluster_by=['order_date'] for Snowflake partition pruning

-- Incremental filter (inside is_incremental() block):
WHERE order_date > (SELECT MAX(order_date) FROM {{ this }})
OR    order_date >= DATEADD('day', -3, CURRENT_DATE())
-- The second line is the late-arriving data lookback window

-- Point-in-time customer FK resolution:
LEFT JOIN customers_snapshot c
  ON c.customer_id = o.customer_id
  AND o.order_date BETWEEN c.dbt_valid_from
                        AND COALESCE(c.dbt_valid_to, '9999-12-31')

-- Result: c.segment = customer's segment WHEN they ordered
-- NOT today's segment — historical accuracy preserved
```

3.5 Testing Strategy

Test Level	Tool	Examples	Failure Action
Schema tests on staging	dbt built-in generics	unique on order_id, not_null on customer_id, accepted_values on status	severity: error — stops pipeline
Range / expression tests	dbt-utils package	expression_is_true: total_amount >= 0, accepted_range on order_date	severity: warn for soft checks, error for critical
Business rule tests	dbt singular tests in tests/ folder	assert_ship_after_order, assert_no_orphan_order_items, assert_revenue_positive	severity: error in prod, warn in dev
Source freshness	dbt source freshness command	warn_after 1 hour, error_after 4 hours on raw_orders	Airflow runs dbt source freshness BEFORE dbt build — fail fast on stale data

CHAPTER 4

Airflow — Pipeline Orchestration

DAG structure · SLA callbacks · Failure recovery

4.1 Why Airflow Over Snowflake Tasks

Capability	Airflow	Snowflake Tasks
Cross-system (S3, Slack, dbt)	Yes — operators for all systems	No — Snowflake SQL only
Per-task SLA callbacks	Yes — <code>sla_miss_callback</code> function	No — only <code>TIMEOUT</code> (kills task)
Visual Gantt chart	Yes — live UI	No — <code>TASK_HISTORY</code> queries only
dbt CLI integration	Yes — <code>BashOperator</code> or <code>DbtRunOperator</code>	No
Retry with backoff	Yes — retries + <code>retry_delay</code> in <code>default_args</code>	Limited
Pool limits (concurrency)	Yes — named pools with slot counts	No — only <code>MAX_CONCURRENCY_LEVEL</code>

4.2 DAG Task Structure — 11 Tasks

Task	Operator	SLA	Key Detail
<code>check_source_freshness</code>	<code>BashOperator</code> → dbt source freshness	15 min	Runs BEFORE dbt build — fail fast if Bronze is stale
<code>refresh_snowpipes</code>	<code>SnowflakeOperator</code> — <code>ALTER PIPE REFRESH</code>	10 min	Catches up any files Snowpipe may have missed
<code>dbt_snapshot</code>	<code>BashOperator</code> → dbt snapshot	30 min	MUST run before staging — SCD2 snapshots are upstream of dims
<code>dbt_run_staging</code>	<code>BashOperator</code> → dbt build path:staging	30 min	type-cast, rename, deduplicate Bronze → staging views
<code>dbt_run_intermediate</code>	<code>BashOperator</code> → dbt build path:intermediate	20 min	Cross-source joins and business logic
<code>dbt_run_dimensions</code>	<code>BashOperator</code> → dbt build path:marts/dimensions	20 min	<code>dim_customers</code> , <code>dim_products</code> , <code>dim_stores</code> , <code>dim_date</code>
<code>dbt_run_facts</code>	<code>BashOperator</code> → dbt build path:marts/facts	60 min	<code>fct_orders</code> , <code>fct_order_items</code> — incremental with PIT FK
<code>dbt_run_gold</code>	<code>BashOperator</code> → dbt build path:marts/gold	30 min	<code>fct_daily_sales</code> , <code>customer_360</code> , RFM scoring
<code>validate_row_counts</code>	<code>SnowflakeOperator</code> — count check on Gold	5 min	Ensures Gold received data for yesterday's date
<code>refresh_materialized_views</code>	<code>SnowflakeOperator</code> — <code>ALTER MATERIALIZED VIEW</code>	5 min	Resumes materialized views after incremental load

notify_success	PythonOperator → Slack API call	2 min	Posts success message with date and row counts
----------------	---------------------------------	-------	--

KEY INSIGHT

Tasks are split by dbt layer so Airflow can: retry just the failed step without rebuilding everything, show exactly wh

4.3 SLA and Failure Callbacks

Callback	When Triggered	Action Taken	Critical Difference
sla_miss_callback	Task ran but exceeded SLA timedelta	Posts Slack warning with task name, DAG ID, elapsed time	Task may still be running or may complete — it's a warning, not a failure
on_failure_callback	Task threw an exception and failed	Posts Slack alert with log URL, marks task FAILED, skips downstream	Pipeline stops here — fix and clear-and-rerun from failed task
on_retry_callback	Task is about to retry after failure	Optional: log retry attempt number for monitoring	Fires before each retry up to the retries count in default_args

IMPORTANT

SLA miss is NOT a failure. A task can exceed its SLA but still complete successfully. Always configure both sla_miss_c

CHAPTER 5

Security and Governance

Masking · Row access · PII tagging · GDPR

5.1 Dynamic Data Masking

Masking policies apply to a column. The column value is hidden or transformed based on the querying role — the row is still returned but the value is masked. Masking is transparent to the querier.

Column	Policy	Admin / Engineer Sees	Senior Analyst Sees	Analyst Sees
customers.email	MASK_EMAIL	Full email	First 2 chars + *** @ domain	*** @ domain
customers.phone	MASK_PHONE	Full phone	Last 4 digits only	Last 4 digits only
customers.credit_limit	MASK_CREDIT	Exact value	Rounded to nearest 1000	NULL
employees.salary	MASK_SALARY	Exact salary	Rounded to nearest 1000	NULL

5.2 Row Access Policy — Territory Filtering

Row access policies filter which ROWS are visible at all. Hidden rows are completely invisible — the user cannot count them or infer their existence. This project uses a mapping table approach: adding a user to a new territory is a single INSERT into USER_TERRITORY_MAP — no DDL changes needed.

Component	Description
USER_TERRITORY_MAP table	Stores: username, territory, valid_from, valid_to (NULL = still active)
Row access policy	RETURNS BOOLEAN — TRUE = row visible. FALSE = row hidden.
Policy condition	EXISTS check in USER_TERRITORY_MAP for CURRENT_USER() and row's territory
Bypass roles	DATA_ENG_ROLE and ACCOUNTADMIN always see all rows
Applying the policy	ALTER TABLE Gold_table ADD ROW ACCESS POLICY policy_name ON (region_id)

Interview Q&A; — How to Explain This Project

These are the questions that come up in every interview where you mention this project. Each answer shows depth — not just 'what' but 'why we chose this' and 'what would break if we did it differently'.

Q1. Walk me through the end-to-end architecture.

HARD

Answer:

Start with the business need: retail company needs daily sales analytics by 7am. 500 stores generate CSV files hourly.

S3 to Snowpipe: files trigger S3 ObjectCreated events via SQS. Snowpipe loads to Bronze within 5 minutes. No credentials stored anywhere — IAM role trust policy.

Bronze to Silver via CDC and dbt: Snowflake Streams detect every row change. We classify each change as new record, SCD2 expire-and-insert, SCD1 overwrite, or soft delete. dbt snapshots implement SCD2. dbt incremental facts use point-in-time FK resolution.

Silver to Gold via dbt: pre-aggregated daily KPIs and Customer 360 with RFM scoring.

Airflow orchestrates 11 tasks in a DAG, runs at 1am daily, SLA callbacks to Slack if any task exceeds 3 hours.

Result: Finance has accurate dashboard data by 5am — two hours ahead of the 7am business SLA.

KEY INSIGHT

Always start with the business outcome, then explain the data flow direction. End with: 'which means stakeholders get

Q2. Why the Medallion Architecture?

MEDIUM

Answer:

Bronze preserves raw data exactly as received. If we find a bug in Silver transforms next month, we reprocess from Bronze — no need to go back to the source system.

Silver applies all business rules once in a version-controlled dbt project. Multiple Gold models read from Silver without duplicating transformation logic.

Gold pre-aggregates what BI tools need. Dashboards query pre-computed aggregates instead of joining 10 tables — query time drops from 30 seconds to under 1 second.

Layers are independently rebuildable: fix a Silver model → dbt run --select affected_model+ rebuilds only the affected downstream chain.

KEY INSIGHT

Medallion = reproducibility (Bronze), governance (Silver), performance (Gold). Each layer serves a different audience.

Q3. Why Snowflake instead of Redshift or BigQuery?

HARD

Answer:

Three concrete reasons for this project:

1. Native CDC via Streams: Snowflake Streams track every DML on Bronze tables without any external CDC tool. The stream offset is transaction-safe — failed tasks don't lose data.

2. Time Travel for debugging: when a bug caused wrong Silver data, we restored with `CREATE TABLE orders_clean CLONE orders BEFORE (STATEMENT => previous_query_id)`. One command, instant recovery.
3. Separated compute and storage: six different warehouses — Snowpipe, transform, analytics, dev — all read the same Bronze data simultaneously with zero contention.

KEY INSIGHT

Give concrete reasons tied to this project's requirements. 'Snowflake is fast' is not an answer. Streams, Time Travel,

Q4. Why dbt snapshots instead of a custom SCD2 stored procedure?**HARD**

Answer:

dbt snapshots give us: automatic `dbt_valid_from` and `dbt_valid_to` columns, built-in change detection via timestamp or column hash strategy, and the snapshot is part of the dbt DAG so `ref()` tracks it as a dependency.

A custom stored procedure would need: detect changes, expire old rows, insert new rows, maintain correct timestamps, handle deletes and reactivations. That is 500+ lines of SQL versus 20 lines of snapshot config. One important gotcha: dbt snapshot runs with the `dbt snapshot` command — separate from `dbt run` or `dbt build`. In Airflow, the snapshot task must run before the staging task, not after.

KEY INSIGHT

`dbt snapshot` = 20 lines. Custom SCD2 proc = 500+ lines. Use the right tool.

Q5. How did you handle late-arriving data in incremental facts?**HARD**

Answer:

Problem: orders placed on Tuesday sometimes arrive in S3 on Wednesday. A naive filter — `WHERE load_date greater than MAX(load_date)` — misses those rows permanently.

Solution: two-part incremental filter. Process rows newer than the max already in the table AND also reprocess the last 3 days as a lookback window.

The lookback window means we reprocess roughly 3× more data on each run but guarantees no late arrivals are ever missed. The `unique_key` on `order_id` ensures the `MERGE` deduplicates any reprocessed rows.

KEY INSIGHT

Lookback window = data integrity insurance. Slightly more work per run, zero missed records.

Q6. What would break if you used `IS_CURRENT=TRUE` instead of PIT FK resolution?**HARD**

Answer:

If we joined facts to `dim_customers` with `IS_CURRENT=TRUE` or `dbt_valid_to IS NULL`:

Every historical fact row would show the customer's `CURRENT` segment — not the segment they had when they placed the order.

Practical impact: customer C001 spent 50,000 dollars as a Consumer from 2021 to 2023. They upgraded to Corporate in 2024. With `IS_CURRENT=TRUE`, all 50,000 shows as Corporate revenue — the Consumer segment loses 50,000 of historical revenue overnight.

This corrupts: segment-level revenue trends, marketing ROI by segment, cohort analysis, and any A/B test involving dimension attributes.

The correct join uses: `event_date BETWEEN dbt_valid_from AND COALESCE(dbt_valid_to, '9999-12-31')`.

KEY INSIGHT

IS_CURRENT=TRUE is correct only for 'what is the customer's segment TODAY?'. For historical analysis it always gives t

Q7. Why Airflow instead of Snowflake Tasks?**HARD**

Answer:

Snowflake Tasks are excellent for Snowflake-native pipelines. We chose Airflow for four reasons specific to this project:

1. Cross-system orchestration: the pipeline touches S3 (check file landing), Snowflake (transforms), and external APIs (Slack, PagerDuty). Airflow has operators for all of these. Snowflake Tasks only run Snowflake SQL.
2. SLA monitoring: Airflow has per-task SLA callbacks that alert before killing a task. Snowflake `USER_TASK_TIMEOUT_MS` kills the task but sends no warning beforehand.
3. Visibility: the Airflow Gantt chart shows every task's start and end time. This is how we found that the facts task was consistently taking 45 minutes and sized up `TRANSFORM_WH` before it became an SLA breach.
4. dbt integration: dbt runs via `BashOperator` or `DbtRunOperator`. Snowflake Tasks cannot invoke the dbt CLI.

Q8. How do you recover when a pipeline task fails?**MEDIUM**

Answer:

1. Alert arrives in Slack with task name, DAG run ID, and log URL.
2. Open Airflow UI, click the failed task, read the error log. If it is a Snowflake error — check Snowflake Query History for the exact SQL. If it is a dbt error — find the compiled SQL in the dbt task log.
3. Fix the root cause. For most failures — stale source data, warehouse OOM, bad data in source — fix then clear-and-rerun from the failed task in Airflow UI. Upstream tasks do not re-run.
4. Streams preserve their offset until consumed. The stream data is still there for the retry — no data is lost when a pipeline task fails.
5. After resolution: document in the ticket, update the runbook, add a monitoring alert if this is a new failure pattern.

KEY INSIGHT

Failed task + Airflow clear-and-rerun = resume from exactly that point. Streams preserve data. No data loss on failure

Interview Cheat Sheet

The 60-Second Pitch

HOW TO INTRODUCE THIS PROJECT

"I built an end-to-end retail analytics platform processing daily sales data from 500 stores. Raw CSV files land in AWS S3 every hour. Snowpipe loads them to Snowflake within 5 minutes.

I used dbt to implement the Silver layer — SCD2 for customer and product dimensions via dbt snapshots, incremental fact tables with point-in-time FK resolution so historical segment analysis is accurate, and a test suite with source freshness checks.

The Gold layer has pre-aggregated daily KPIs and a Customer 360 with RFM scoring. Apache Airflow orchestrates the 11-task pipeline at 1am daily with SLA callbacks to Slack.

Key Design Decisions

Decision	Alternative	Why This Was Better	Interview Impact
Row hash at COPY INTO time	Timestamp watermark only	Source files have no updated_at — hash is the only reliable change detector	Shows real-world constraint awareness
dbt snapshot for SCD2	Custom Snowflake MERGE procedure	20 lines of config vs 500+ lines of SQL — version-controlled and auto-tested	Shows tool-fit judgment not habit
PIT FK in facts	IS_CURRENT=TRUE join	IS_CURRENT gives TODAY's segment for ALL historical orders — always wrong	Most candidates miss this — answering correctly is a differentiator
Incremental with lookback	Append-only no lookback	Late arrivals (Tuesday records arriving Wednesday) would be permanently lost	Shows production data quality experience
Airflow over Snowflake Tasks	Native Task DAG	Cross-system orchestration, SLA callbacks, dbt CLI all require Airflow	Shows ability to pick the right tool
Transient Bronze tables	Permanent tables	Bronze is reproducible from S3 — Fail-safe overhead is wasted cost	Shows cost and table-type awareness

Numbers to Know

Metric	Value	Why It Matters
Source tables	5, approx 200K rows per day each	Sets scale context for the interviewer
Snowpipe latency	Under 5 minutes file-to-table	Answer to: how fresh is Bronze?
dbt models	~20 (5 staging, 3 intermediate, 12 marts)	Shows realistic project scope
dbt tests	~35 schema + singular tests	Shows testing discipline

Airflow tasks	11 tasks, 1 DAG	Shows proper dependency graph design
Pipeline SLA	Gold ready by 5am, business needs it by 7am	Shows SLA awareness
Warehouse credits	Approx 150 per month	Shows cost awareness
SCD types used	0 for date, 1 for stores, 2 for customers and products	Shows SCD knowledge applied